Task 1 (Mixed Topics) [16 points]

Fill in the gaps. If not specified in a different way, each correct entry gives 1 point, and each wrong or omitted entry gives 0 points.

(a) Big-O notation (5 p.)

In this subtask, each correct entry gives 1/2 point. Let $f: \mathbb{N} \to \mathbb{R}^+$ and $g: \mathbb{N} \to \mathbb{R}^+$ be functions.

- We say "$g(n) = \Omega(f(n))$" if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

  $g(n) \geq c \cdot f(n)$ for all $n \geq n_0$.

- *Limit rule:* If $\lim_{n\to\infty} \frac{g(n)}{f(n)} = c$ for some $c \geq 0$, then $g(n) = O(f(n))$.

  Domination rule: If $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$, then $g(n) \neq f(n)$.

- If $f(n) = \frac{(n^3 - n^2) \cdot 8^{n/3}}{7n^4 + 5n\sqrt{n}}$, then $f(n) = \Theta(n^a \cdot b^n)$ for constants $a = 1$ and $b = 2$.

- Let $n \in \mathbb{N}$. Then $\sum_{i=0}^{n} i = \Theta(n^2)$ and $\sum_{i=1}^{n} \frac{1}{i} = \Theta(\log n)$.

- If $f(n) = 5 - 2\sin(n)$, then $f(n) = O(1)$ and $f(n) = \Omega(1)$.

(b) Modular arithmetic (6 p.)

Let $x, y, N \in \mathbb{N}$ be n-bit integers such that $N \geq 2$.

- The value $(x \cdot y) \bmod N$ can be computed naively using grade school methods.

  The number of bit operations when using these methods is $O(n^2)$.

- If $x < N$, the value $x^y \bmod N$ can be computed using the following recursive formula:

$$x^y \bmod N = \begin{cases} 1 & \text{, if } y = 0 \\ x & \text{, if } y = 1 \\ (x^2 \bmod N)^{\lfloor y/2 \rfloor} \bmod N & \text{, if } y \geq 2 \text{ even} \\ ((x^2 \bmod N)^{\lfloor y/2 \rfloor} \bmod N \cdot x) \bmod N & \text{if } y \geq 2 \text{ odd} \end{cases}$$

  The number of bit operations when using this method is $O(n^3)$.

- If $x \geq y$, the value $\gcd(x, y)$ can be computed using the following recursive formula:

$$\gcd(x, y) = \begin{cases} x & \text{, if } y = 0 \\ \gcd(y, x \bmod y) & \text{, if } y \geq 1 \end{cases}$$

  The number of bit operations when using this method is $O(n^2)$.

Continued on next page!

(c) Divide-and-Conquer algorithms (5 p.)

- Karatsuba's algorithm multiplies two n-bit integers in time $O(\ldots n^{1.585} \ldots)$.

  When the algorithm is carried out, it is called recursively $\ldots 3 \ldots$ times (on the first level only).

- Randomized Quickselect finds the element of rank k of n elements in expected time $O(\ldots n \ldots)$.

- Randomized Quicksort sorts n distinct elements in expected time $O(\ldots n \log n \ldots)$.

- If $\mathcal{A}$ is a comparison-based sorting algorithm, it requires time $\Omega(\ldots n \log n \ldots)$ (worst case) to sort n elements.

Task 2 (Primality Testing) [8 points]

(a) Formulate the Fermat test: (2 p.)

Input: Odd integer $N \geq 3$
Method:

1 choose integer a ... at random ... from $\{1, 2, 3, \ldots, N-1\}$;
2 calculate integer $b \leftarrow a^{N-1} \bmod N$;
3 **if** $b \neq 1$ **then return** 1 **else return** 0;

(b) Describe the **output behavior** of the Fermat test: (2 p.)

- If N is prime, then the output is always 0.
- If N is a composite non-Carmichael number, then $\Pr(\text{output is } 0) \leq \frac{1}{2}$.
  (Possible answers: "$< c$", "$\leq c$", "$= c$", "$\geq c$", or "$> c$" for some appropriate $0 \leq c \leq 1$.)

(c) Let $N \geq 9$ be some composite odd number. An integer $1 \leq a < N$ is called **"bad"** if (3 p.)
$a^{N-1} \bmod N \neq 1$ and **"good"** otherwise.
N is called a **Carmichael number** if

.....

.....

The (iterated) Fermat test does not behave well on Carmichael numbers. To resolve this problem, one can use the Miller test.

(d) The expected running time for generating a random n-bit prime with error probability $\leq 4^{-t}$ is (1 p.)
$O(n^3 \cdot t)$.

Task 3 (RSA) [6 points]

We consider the RSA cryptosystem as discussed in class. In the following three parts, fill in the missing words or formulas!

(a) Preprocessing/Key generation (4 p.)

We randomly choose two distinct primes p and q, define $N = p \cdot q$, and calculate $\varphi(N) =$ $(p-1)(q-1)$.

For Bob's public key (N, e), we choose e, $3 \le e < \varphi(N)$, such that $\gcd(e, \varphi(N)) = 1$.

Bob's private key is (N, d), where d, $3 \le d < \varphi(N)$, is such that $e \cdot d \bmod \varphi(N) = 1$.

d can be calculated using the Euclidean algorithm.

(b) Communication (1 p.)

Alice has a message/plaintext $x \in [N] = \{0, 1, \ldots, N-1\}$. She computes $y =$ $x^e \bmod n$ and sends y to Bob. Bob receives y and computes $z =$ $y^d \bmod n$.

(c) Correctness claim/Decryption condition (1 p.)

$(x^e \bmod N)^d \bmod N = x$ for all $x \in [N]$.

(c) Let $G = (V, E)$ be a digraph with $|V| = n$ and $|E| = m$. For a node $v \in V$, outdeg(v) is the number of outgoing edges $(v, w) \in E$ and indeg(v) is the number of ingoing edges $(u, v) \in E$. What is the **time** needed to compute these numbers if G is given in one of the following representations?[1] (6 p.)

	Adjacency matrix	Adjacency lists	Adjacency array
outdeg(v):	$O(n)$	$O(n + m)$	$O(n)$
indeg(v):	$O(n)$	$O(n + m)$	$O(m)$

[1] In general, $O(n + m)$ is not the same as $O(n)$ or $O(m)$. A bound $O(\text{outdeg}(v))$ can be much smaller than $O(n)$ and $O(m)$.

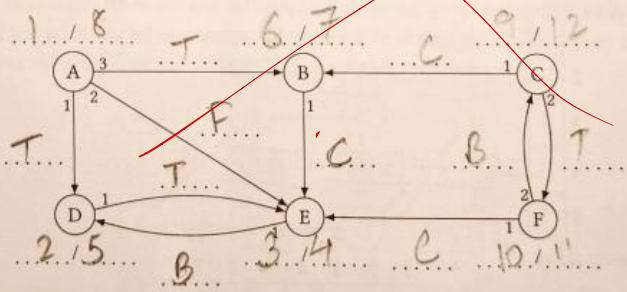
n^2 $n + m$ $m + n$

n^2 $n + m$ $m + n$

Task 6 (Depth-First Search – DFS) [11 points]

(a) Let $G = (V, E)$ be a digraph with $n = |V|$ nodes and $m = |E|$ edges. **DFS** assigns each node two numbers called Pre / Post, and each edge is labeled with one of the following edge labels: Tree, forward, backward, cross. The running time is O(m log n). (3 p.)

(b) Perform **depth-first search** on the following digraph. The nodes are ordered alphabetically, i. e., start with node A. The numbers on outgoing edges of a node indicate the ordering of its neighbors in its adjacency list. (6 p.)



(c) A **topological ordering** of G is an ordering $(v_1, \ldots, v_n)$ of the nodes such that (2 p.)

Given the result of a DFS run in G, a topological ordering of G can be computed as follows (if it exists):

Task 7 (Shortest Paths – Dijkstra's Algorithm) [9 points]

(a) Dijkstra's algorithm can be applied when a weighted digraph $G = (V, E, c)$ and a start node $s \in V$ are given. It calculates the distances $d(s, v)$, for all $v \in V$. All edge weights $c(u, v)$, $(u, v) \in E$, must be Positive Real number. The running time of Dijkstra's algorithm (using a priority queue, implemented as a binary heap) is O(n+m log n), where $n = |V|$ and $m = |E|$. (2 p.)

Task 8 (Minimum Spanning Trees – Jarník/Prim and Kruskal's Algorithm) [11 points]

In this task, $G = (V, E, c)$ is a connected (undirected) graph with edge weights $c(e)$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

(a) A **minimum spanning tree** (MST) for G is an edge set $T \subseteq E$ such that (2 p.)

Cost smallest
Non-cyclic

Chosen edges: (illegible)

(d) The Jarník/Prim algorithm finds a minimum spanning tree in time O(m log n) using a priority queue, realized as a binary heap. (1 p.)

Kruskal's algorithm finds a minimum spanning tree in time O(m log n) using the union-find data structure, realized as trees. (1 p.)

Task 1 (Mixed Topics) [18 points]

Fill in the gaps. If not specified in a different way, each correct entry gives 1 point, and each wrong or omitted entry gives 0 points.

(a) Big-O notation (4 p.)

In this subtask, each correct entry gives 1/2 point. Let $f: \mathbb{N} \to \mathbb{R}^+$ and $g: \mathbb{N} \to \mathbb{R}^+$ be functions.

- We say "$g(n) = O(f(n))$" if there are $c > 0$ and $n_0 \in \mathbb{N}$ such that

  $c \cdot f(n) \geq g(n)$ for all $n > n_0$

- *Limit rule:* If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$ then $f(n) = O(g(n))$

- If $f(n) = \frac{(n^4+1)\cdot 3^{2n}}{n^2 + 5\sqrt{n}}$, then $f(n) = \Theta(n^a \cdot b^n)$ for constants $a =$ 2 and $b =$ 9.

- Let $n \in \mathbb{N}$. Then $\sum_{i=0}^{n} i = \Theta($ n^2 $)$ and $\sum_{i=0}^{n} 2^i = \Theta($ n $)$

(b) Modular arithmetic (4 p.)

- For $x, y, N \in \mathbb{Z}$, $N \geq 2$, "$x \equiv y \pmod N$" means $x \bmod N = y \bmod N$

- For nonnegative n-bit integers x, y, N, $N \geq 2$, $x < N$, the value $x^y \bmod N$ can be computed using the following recursive formula:

$$x^y \bmod N = \begin{cases} 1 & \text{, if } y = 0 \\ x & \text{, if } y = 1 \\ (x^2 \bmod N)^{y/2} \bmod N & \text{, if } y \geq 2 \text{ even} \\ ((x^2 \bmod N)^{y/2} \bmod N \cdot x) \bmod N & \text{, if } y \geq 2 \text{ odd} \end{cases}$$

The number of bit operations when using this method is $O($ n^3 $)$.

(c) Divide-and-Conquer algorithms (5 p.)

- Karatsuba's algorithm multiplies two n-bit integers in time $O($ $n^{1.585}$ $)$.

  When the algorithm is carried out, it is called recursively 3 times (on the first level only).

- Mergesort sorts n elements in time $O($ $n \log n$ $)$ (worst case).

- Randomized Quicksort sorts n distinct elements in time $O($ n^2 $)$ (worst case) and expected time $O($ $n \log n$ $)$.

dominating rule

$\lim \frac{f(n)}{g(n)} = 0$

$f(n) + g(n)$

$h < i$: $\Theta(i)$

$q = 1$ $\Theta(n)$

$h > 1$ $\Theta(q^n)$

$\frac{n(n+1)}{2}$

$\Theta(2^n)$

all both serve in merge

$(a^2 \bmod N)^{\lfloor N/2 \rfloor} \bmod N$

$\left(\left[a^2 \bmod N\right]^{\lfloor N/2 \rfloor} \bmod N \cdot a\right) \bmod N$

Task 2 (Primality Testing) [7 points]

(a) Formulate the Fermat test: (2 p.)

Input: Odd integer $N \geq 3$
Method:

1 choose a at random from $\{1, \ldots, N-1\}$;
2 $b \leftarrow a \bmod N$;
3 **if** $b \neq 1$ **then return** 1 **else return** 0;

(b) Describe the behavior of the Fermat test: (2 p.)

- If N is prime, then output is 0.
- If N is a composite non-Carmichael number, then $\Pr(\ldots \leq 1/2 \ldots)$.

(c) The iterated Fermat test returns 0 if and only if ℓ independent repetitions of the simple Fermat test return 0. If N is a composite non-Carmichael number, its behavior is as follows: (1 p.)

$\Pr(\text{output } 0) \leq \frac{1}{2^\ell}$

(d) A composite odd number $N \geq 9$ is called a Carmichael number if (1 p.)

$a^{N-1} \bmod N = 1$ for all a with $\gcd(a, N) = 1$ for all a relatively prime to N

(e) Consider the following procedure:

Randomly pick an n-bit number N until N is prime.

On average this procedure will halt within $O(n)$ rounds. **(1 p.)**

(d) Dijkstra's algorithm (5 p.)

- Dijkstra's algorithm computes all distances $d(s, v)$ from a node $s \in V$ in a digraph $G = (V, E)$ with edge weights $\ell(u, v)$, for $(u, v) \in E$. All edge weights must be non negative. The running time of Dijkstra's algorithm is $O(n + m \log n)$.
- Dijkstra's algorithm uses a priority queue (PQ). In each round, the PQ operation extract minimum determines the next node u to be scanned. While examining all outgoing edges $(u, v) \in E$, the PQ operation insert is executed if a new node v is found. If node v is not new, maybe the PQ operation decrease key is executed.

Task 4 (Recurrence Relations) [7 points]

(a) Consider some algorithm A. It solves instances of size $n = 1$ in constant time. If $n > 1$, it divides the input into 5 sub-instances of size $\lfloor n/2 \rfloor$ in time $O(n^2)$, solves these subproblems recursively, and combines the solutions to obtain a solution of the original input in time $O(n \log n)$.

Let $T(n)$ be the running time of A on inputs of size $n \geq 1$. It fulfills the following recurrence relation: (2 p.)

$$T(n) \leq \begin{cases} \dots & \text{, if } n = 1 \\ \dots & \text{, if } n > 1 \end{cases}$$

(b) Consider some algorithm B. It solves problems of size $n \geq 1$ in time $T(n)$, where

$$T(n) \leq \begin{cases} 1 & \text{, if } n = 1 \\ 4 \cdot T(\frac{n}{2}) + n^2 & \text{, if } n > 1 \end{cases}$$

Solve this recurrence relation using the Master theorem. What are the values of the important constants a, b, and d? Which case of the Master theorem applies? (3 p.)

(c) Consider some algorithm C. It solves problems of size $n \geq 1$ in time $T(n)$, where

$$T(n) = \begin{cases} 1 & \text{, if } n = 1 \\ 2 \cdot T(n-1) + 1 & \text{, if } n > 1 \end{cases}$$

Solve this recurrence relation exactly without big-O notation. (You will need a direct calculation. The Master theorem does not help here.) (2 p.)

(b) Assume letters $s_1, s_2, \dots, s_n$ with corresponding frequencies $f_1, f_2, \dots, f_n$ are given.

If s_i and s_j are "two least frequent letters", there is an optimal coding tree in which

.....

outdegree - O(1) Matrix O(n)

indegree

O(n+m)

O(m)

O(n)

(c) Let $G = (V, E)$ be a digraph with $|V| = n$ and $|E| = m$. For a node $v \in V$, outdeg(v) is the number of outgoing edges $(v, w) \in E$. What is the time needed to compute outdeg(v) if G is given in one of the following representations? (3 p.)

Adjacency matrix	Adjacency lists	Adjacency array
O(n)	O(n + m)	O(n)

(b) A *topological ordering* of a digraph $G = (V, E)$ is an ordering $(v_1, \ldots, v_n)$ of the nodes such that (1 p.)

all edges point from left to right

A topological ordering of G exists if and only if G has no cycles. Only considering the result of a DFS run in G, how can we decide if G has a cycle or not? (1 p.)

No back edge exist in G

If a topological ordering of G exists, one can be computed, given the result of a DFS run in G, as follows: (1 p.)

decreasing order

(c) What are the *strongly connected components* of the digraph G depicted in (a)? (Give the node sets.) (1 p.)

{ {A, B, D}, {E}, {C, F} }

How does Kosaraju's algorithm compute strongly connected components of a digraph G? (3 p.)

(1) Run DFS(G) finding post(v) for all node of G

(2) Calculate by reversing all the edge

(3) Run DFS(G^R) Now the node are checked in decreasing post number from (1)

(4) Output: output the nodes of DFS tree

What is the running time of Kosaraju's algorithm on input $G = (V, E)$ where $|V| = n$ and $|E| = m$? (1 p.)

O(n+m)

Task 7 (Minimum Spanning Trees – Kruskal's Algorithm) [9 points]

In this task, $G = (V, E)$ is a connected (undirected) graph with edge weights $w(e)$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

(a) A minimum spanning tree in G is an edge set $T \subseteq E$ with the following properties: (2 p.)

(1) cost of the tree would be minimum

(2) The edge should not be cycle

(c) In the situation of **(b)**, consider the next round: Which edge is checked now? Which operations are executed on the union-find data structure and what are their results (e. g., "find(X) returns Y")? (The results should be consistent with the state of the union-find data structure in your answer for **(b)**.)(3 p.)

.....

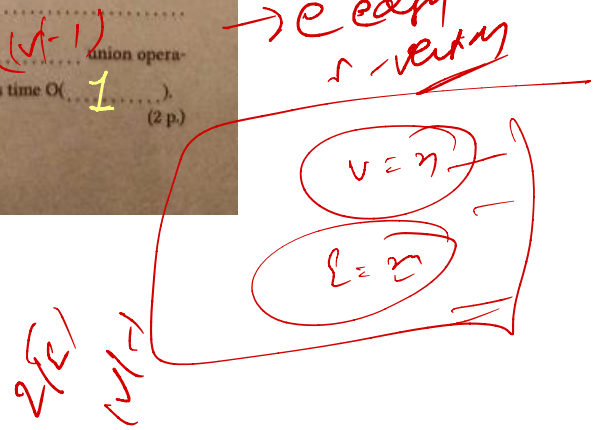
.....

.....

.....

.....

(d) Overall, Kruskal's algorithm needs exactly $2|E|$ find operations and $(|V|-1)$ union operations. A find operation costs time $O(\log n)$, and a union operation costs time $O(1)$. (2 p.)



Task 1 (Mixed Topics) [16 points]

Fill in the gaps. If not specified in a different way, each correct entry gives 1 point, and each wrong or omitted entry gives 0 points.

(a) Big-O notation (5 p.)

In this subtask, each correct entry gives 1/2 point. Let $f: \mathbb{N} \to \mathbb{R}^+$ and $g: \mathbb{N} \to \mathbb{R}^+$ be functions.

- We say "$g(n) = \Omega(f(n))$" if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

  $c \cdot f(n) \leq g(n)$ for all $n \geq n_0$.

- *Limit rule:* If $\lim_{n\to\infty} \frac{g(n)}{f(n)} = c$ for some $c \geq 0$, then $g(n) =$ $O(f(n))$.

  Domination rule: If $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$, then $f(n) + g(n) = O(f(n))$.

- If $f(n) = \frac{(n^4 + n^2) \cdot 8^{n/3}}{7n^5 - 4n\sqrt{n}}$, then $f(n) = \Theta(n^a \cdot b^n)$ for constants $a =$ -1 and $b =$ 2. −1

- Let $n \in \mathbb{N}$. Then $\sum_{i=0}^{n} i = \Theta($ n^2 $)$ and $\sum_{i=1}^{n} \frac{1}{i} = \Theta($ $\ln n$ $)$.

- If $f(n) = 7 - 3\sin(n)$, then $f(n) = O($ 1 $)$ and $f(n) = \Omega($ 1 $)$.

(b) Modular arithmetic (6 p.)

Let $x, y, N \in \mathbb{N}$ be n-bit integers such that $N \geq 2$.

- The value $(x \cdot y)$ mod N can be computed naively using grade school methods.

  The number of bit operations when using these methods is $O($ N^2 $)$.

- If $x < N$, the value x^y mod N can be computed using the following recursive formula:

$$x^y \bmod N = \begin{cases} 1 & \text{, if } y = 0 \\ x & \text{, if } y = 1 \\ (x^2 \bmod N)^{\lfloor y/2 \rfloor} \bmod N & \text{, if } y \geq 2 \text{ even} \\ ((x^2 \bmod N)^{\lfloor y/2 \rfloor} \bmod N \cdot x) \bmod N & \text{, if } y \geq 2 \text{ odd} \end{cases}$$

  The number of bit operations when using this method is $O($ n^3 $)$.

- If $x \geq y$, the value $\gcd(x, y)$ can be computed using the following recursive formula:

$$\gcd(x, y) = \begin{cases} x & \text{, if } y = 0 \\ \gcd(y, x \bmod y) & \text{, if } y \geq 1 \end{cases}$$

  The number of bit operations when using this method is $O($ n^2 $)$.

Continued on next page!

(c) Divide-and-Conquer algorithms (5 p.)

- Karatsuba's algorithm multiplies two n-bit integers in time $O(\ldots n^{\log_2 3} \ldots)$.
  When the algorithm is carried out, it is called recursively ...3... times (on the first level only).
- Randomized Quickselect finds the element of rank k of n elements in expected time $O(\ldots n \ldots)$.
- Randomized Quicksort sorts n distinct elements in expected time $O(\ldots n \log n \ldots)$.
- If $\mathcal{A}$ is a comparison-based sorting algorithm, it requires time $\Omega(\ldots n \log n \ldots)$ (worst case) to sort n elements.

• Mergesort sorts n element in time O(nlog n) worst case

Task 2 (Primality Testing with the Fermat Test) [8 points]

(a) Formulate the Fermat test: (4 p.)

Input: Odd integer $N \geq 3$
Method:

1 choose integer a ...at random... from $\{1, 2, \ldots, N-1\}$;
2 calculate integer $b \leftarrow a^{N-1} \ldots \text{mod } N \ldots$;
3 **if** ...$b \neq 1$... **then return** 1 **else return** 0;

(b) Describe the **output behavior** of the Fermat test: (2 p.)

- If N is prime, then the output is always ...0...
- If N is a composite non-Carmichael number, then $\Pr(\text{output is } 0)$ ...$< 1/2$...
  (E. g. "$= \frac{1}{3}$" or "$\geq \frac{5}{7}$" or "< 1" or ...)

(c) Let $N \geq 9$ be some composite odd number. N is called a **Carmichael number** if (1 p.)

$a^{N-1} \text{ mod } N = 1$ for all a with $\gcd(a, N) = 1$
...f. for all a relatively prime to N...

(d) The expected running time for generating a random n-bit prime with error probability $\leq 4^{-\ell}$ is (1 p.)

$O(\ldots)$. $O(n^3 \cdot \ell)$

Task 8 (Minimum Spanning Trees – Jarník/Prim and Kruskal's Algorithm) [10 points]

In this task, $G = (V, E, c)$ is a connected (undirected) graph with edge weights $c(e) \in \mathbb{R}$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

(a) Let $T \subseteq E$ be a set of edges in G. (3 p.)

(i) T is a **spanning tree** for G if ... V, T is a tree ...

(ii) The **weight** of T is $c(T) :=$ ... $\sum_{e \in E} c(e)$ ... $\sum_{i=1}^{n} c_i(t)$ ...

(iii) T is a **minimum spanning tree** for G if ...

...

(d) The Jarník/Prim algorithm finds a minimum spanning tree in time $O($ ... $m \log(n)$ ... $)$ using a priority queue, realized as a binary heap. (1 p.)

Kruskal's algorithm finds a minimum spanning tree in time $O($ ... $m \log n$ ... $)$ using the union-find data structure, realized as trees. (1 p.)

Task 1 (Mixed Topics) [16 points]

Fill in the gaps. If not specified in a different way, each correct entry gives 1 point, and each wrong or omitted entry gives 0 points.

(a) Big-O notation (5 p.)

In this subtask, each correct entry gives 1/2 point. Let $f: \mathbb{N} \to \mathbb{R}^+$ and $g: \mathbb{N} \to \mathbb{R}^+$ be functions.

- We say "$g(n) = \Omega(f(n))$" if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that
  $c \cdot g(n) \geq f(n)$ for all $n \geq n_0$.
- *Limit rule:* If $\lim_{n\to\infty} \frac{g(n)}{f(n)} = c$ for some c, then $g(n) = O(f(n))$.
- *Domination rule:* If $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$, then $g(n) + f(n) = O(f(n))$.
- If $f(n) = \frac{(n^5+1)\cdot 9^{n/2}}{4n^3 + 5\sqrt{n}}$, then $f(n) = \Theta(n^a \cdot b^n)$ for constants $a =$ 2 and $b =$ 3.
- Let $n \in \mathbb{N}$. Then $\sum_{i=0}^{n} i = \Theta($ n^2 $)$ and $\sum_{i=0}^{n} 2^i = \Theta($ 2^n $)$.
- If $f(n) = 2 + \sin(n)$, then $f(n) = O($ 1 $)$ and $f(n) = \Omega($ 1 $)$.

(b) Modular arithmetic (6 p.)

Let $x, y, N \in \mathbb{N}$ be n-bit integers such that $N \geq 2$.

- The value $(x \cdot y) \bmod N$ can be computed naively using grade school methods.
  The number of bit operations when using these methods is O( ).
- If $x < N$, the value $x^y \bmod N$ can be computed using the following recursive formula:

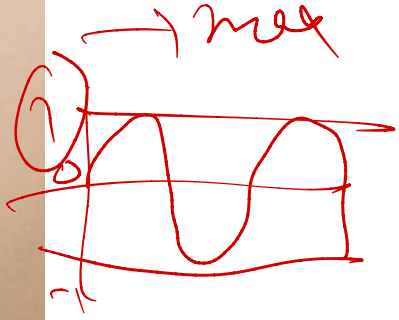
$$x^y \bmod N = \begin{cases} 1 & \text{, if } y = 0 \\ x & \text{, if } y = 1 \\ (x^2 \bmod N)^{\lfloor y/2 \rfloor} \bmod N & \text{, if } y \geq 2 \text{ even} \\ ((x^2 \bmod N)^{\lfloor y/2 \rfloor} \bmod N) \cdot x \bmod N & \text{, if } y \geq 2 \text{ odd} \end{cases}$$

  The number of bit operations when using this method is O( ).
- If $x \geq y$, the value $\gcd(x, y)$ can be computed using the following recursive formula:

$$\gcd(x, y) = \begin{cases} x & \text{, if } y = 0 \\ \gcd(y, x \bmod y) & \text{, if } y \geq 1 \end{cases}$$

  The number of bit operations when using this method is O( ).

Continued on next page!



(c) Divide-and-Conquer algorithms (5 p.)

- Karatsuba's algorithm multiplies two n-bit integers in time O($n^{1.585}$).
  When the algorithm is carried out, it is called recursively 3 times (on the first level only).
- Mergesort sorts n elements in time O($n \log n$) (worst case).
- Randomized Quicksort sorts n distinct elements in expected time O($n \log n$).
- Randomized Quickselect finds the element of rank k of n elements in expected time O(n).

Task 4 (Recurrence Relations) [7 points]

(a) Consider some algorithm A. It solves instances of size $n = 1$ in constant time. If $n > 1$, it divides the input into 5 sub-instances of size $\lfloor n/2 \rfloor$ in time $O(n^3)$, solves these subproblems recursively, and combines the solutions to obtain a solution of the original input in time $O(n^2 \log n)$.

Let $T(n)$ be the running time of A on inputs of size $n \geq 1$. It fulfills the following recurrence relation: (2 p.)

$$T(n) \leq \begin{cases} \ldots\ldots\ldots\ldots\ldots\ldots & \text{, if } n = 1 \\ \ldots\ldots\ldots\ldots\ldots\ldots & \text{, if } n > 1 \end{cases}$$

(b) Consider some algorithm B. It solves problems of size $n \geq 1$ in time $T(n)$, where

$$T(n) \leq \begin{cases} 4 & \text{, if } n = 1 \\ 8 \cdot T(\frac{n}{2}) + 5 \cdot n^3 - n & \text{, if } n > 1 \end{cases}$$

Solve this recurrence relation using the **Master theorem**. What are the values of the important constants a, b, and d? Which case of the Master theorem applies? (3 p.)

(c) Consider some algorithm C. On input $n \geq 1$ it calculates $f(n)$, where

$$f(n) = \begin{cases} 1 & \text{, if } n = 1 \\ 2 \cdot f(n-1) + 1 & \text{, if } n > 1 \end{cases}$$

Solve this recurrence relation **exactly without big-O notation**. (You will need a direct calculation. The Master theorem does not help here.) (2 p.)

Task 8 (Minimum Spanning Trees – Kruskal's Algorithm) [11 points]

In this task, $G = (V, E)$ is a connected (undirected) graph with edge weights $w(e)$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

(a) A **minimum spanning tree** (MST) in G is an edge set $T \subseteq E$ with the following properties: (2 p.)

(1) .....

(2) .....

(c) $X \subseteq E$ is called **extendible** if there is an MST T such that $X \subseteq T$ (1 p.)

Formulate the **cut property**: (2 p.)

(d) **Kruskal's algorithm** needs exactly 2·e find operations and V−1 union operations. A find operation costs time O(log n), and a union operation costs time O(1). (2 p.)

Task 1 (Mixed Topics) [12 points]

Fill in the gaps.

(a) Big-O Notation (5 p.)

Let $f: \mathbb{N} \to \mathbb{R}^+$ and $g: \mathbb{N} \to \mathbb{R}^+$ be functions.

- We say "$g(n) = O(f(n))$", if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that $f(n) \geq g(n)$ for all $n \geq n_0$
- We say "$g(n) = \Omega(f(n))$", if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that $g(n) \geq f(n)$ for all $n \geq n_0$
- We say "$g(n) = \Theta(f(n))$" if $g(n) = O(f(n))$ and $g(n) = \Omega(f(n))$
- If $f(n) = \frac{(n^2\sqrt{n} + n^2 \log n) \cdot 3^{n \log_3 2}}{3n + 2\sqrt{n}}$, then $f(n) = \Theta(n^a b^n)$ for constants $a =$ 0.5 and $b =$ 2.
- If $f(n) = n^2 \sin^2(n) + n^2 + \sqrt{n}$, then $f(n) = O(n^2)$ and $f(n) = \Omega(n^2)$.

(b) Divide-and-Conquer Algorithms (4 p.)

- Karatsuba's algorithm multiplies two n-bit integers in time $O(n^{1.585})$.
  When the algorithm is executed, it calls itself 3 times at the first level of recursion.
- Mergesort sorts n elements in time $O(n \log n)$ in the worst case.
- Randomized Quicksort sorts n distinct elements in expected time $O(n \log n)$, if the input elements are distinct.

(c) Greedy Algorithms (3 p.)

- A greedy algorithm incrementally constructs the solution for a problem instance by repeatedly splitting the instance into smaller instances of equal size.
  Is this statement true? False
- Dijkstra's algorithm computes all distances $d(s, v)$ from a vertex $s \in V$ in a digraph $G = (V, E)$ with edge weights $c(u, v)$, for $(u, v) \in E$. All edge weights must be non negative
- The greedy event scheduling algorithm repeatedly chooses, from among the selectable events, an event that has earlier finish time / shortest duration / least conflict

Shortest finish time and call it to plan

finish time

Task 2 (Recurrence Relations for Divide-and-Conquer Algorithms) [7 points]

(a) Consider an algorithm with the following running-time behaviour. The algorithm solves problem instances of size up to $n = 100$ in constant time. For instances of size $n > 100$, it divides the instance into 3 sub-instances of size $\lfloor n/5 \rfloor$ in time $O(n \log n)$, solves these recursively, and combines the partial solutions in time $O(n)$. Write down a recurrence relation for the running time $T(n)$ of the algorithm. (2 p.)

$$T(n) \leq \begin{cases} 1, & \text{if } n \leq 100 \\ 3 \cdot T\left(\frac{n}{5}\right) + c \cdot n \log n, & \text{if } n > 100 \end{cases}$$

(b) The following relations describe the running time $T(n)$ of an algorithm on inputs of size $n \geq 1$.

$$T(n) \leq \begin{cases} 1, & \text{if } n = 1 \\ 7 \cdot T\left(\frac{n}{2}\right) + 10n^2, & \text{if } n > 1 \end{cases}$$

$a = b^d \Rightarrow O(n^d \log n)$
$a > b^d \Rightarrow O(n^{\log_b a})$
$a < b^d \Rightarrow O(n^d)$

Using the above relations, obtain a closed-form expression (in big-O notation) for the running time of the algorithm. (2 p.)

$O(n^{\log_2 7})$

(c) Consider the following recurrence relation defining a function f over natural numbers $n \geq 1$.

$$f(n) = \begin{cases} 1, & \text{if } n = 1 \\ 2^{n-1} + f(n-1) + n, & \text{if } n > 1 \end{cases}$$

Solve this recurrence relation **exactly** to obtain a closed-form expression. (3 p.)

Level		# no of calls	Merge cost	Total cost
0	n	1	$2^{n-1} + n$	
1	$n-1$	1	$2^{n-2} + n-1$	
2	$n-2$	1	$2^{n-3} + n-2$	
⋮				
d	$n-d$	1	$2^{n-d-1} + n-d$	$2^{n-d-1} + n-d$
⋮				
$n-d=1$, $d=n-1$	1	1	1	1

$\sum_{d=0}^{n-1} 2^{n-d-1} + n-d$

Task 3 (Problem Descriptions) [12 points]

Complete the following descriptions of various computational problems by filling in the blanks.

(a) Problem: SELECTION PROBLEM (3 p.)

Input: • Array int [1...n] and k-th element $1 \le k \le n$

•

Output: k-th smallest element in the list if value x

(b) Problem: EDIT DISTANCE (3 p.)

Input: Integer A[1...n], B[1...n]

Output: Edit distance of A B

The *edit distance* of two strings A, B is defined as the minimum no. of character insertion, deletion, substitution to convert one to other

(c) Problem: MINIMUM SPANNING TREE (3 p.)

Input: weighted graph $G(V,E,C)$ where $V(1...n) \subseteq V$

Output: M.S.T of G

(d) Problem: LONGEST INCREASING SUBSEQUENCE (3 p.)

Input: Array of integers [1...n]

Output: length L of the longest seq

A *longest increasing subsequence* of a sequence of integers $a_1, \ldots, a_n$ is defined as

a seq $a_{i_1} \ldots a_{i_l}$ of maximal length such that $i_1 < \ldots < i_l$

$a_{i_1} < \ldots < a_{i_l}$

Task 5 (Minimum Spanning Trees) [10 points]

In this task, $G = (V, E, c)$ is a connected (undirected) graph with edge weights $c(e) \in \mathbb{R}$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

(a) Let $T \subseteq E$ be a set of edges in G. (3 p.)

(i) T is a **spanning tree** for G if acyclic and all nodes are traversed

(ii) The **weight** of T is $c(T) = \sum_{i=1} c_i(T)$

(iii) T is a **minimum spanning tree** for G if Total cost is minimum among all other spanning tree

(d) Prim's algorithm finds a minimum spanning tree in time O($m \log n$) using a priority queue, realized as a binary heap. (1 p.)

Kruskal's algorithm finds a minimum spanning tree in time O($m \log n$) using the union-find data structure, realized using trees with the union-by-rank rule. (1 p.)

Task 6 (Shortest Paths) [9 points]

$V + E \log n$

(a) Dijkstra's algorithm can be applied when a weighted digraph $G = (V, E, c)$ and a start node $s \in V$ are given. It calculates the distances $d(s, v)$, for all $v \in V$. The running time of Dijkstra's algorithm (using a priority queue, implemented as a binary heap) is O($n + m \log n$), where $n = |V|$ and $m = |E|$. (2 p.)

(b) What is the purpose of Huffman's algorithm? Describe what it takes as input and the kind of object it outputs. (2 p.)

Coding of letters / Symbols with binary code s.t. no code is prefix of another and frequent letters have a short code.

Inp: A finite alphabet A ($|A| > 2$) and relative frequency $P : A \to [0,1)$ with $\sum_{a \in A}$

Out: $P \to P$ & code $C : A \to \{0,1\}^*$ for A with minimum Cost $\sum_{a \in A} P(a) \cdot |C(a)|$